



can you create a .md file linking how GPU system limits such as those in attached screenshot impact possible parallelization.

Below is a .md document you can save as `gpu_parallelization_limits.md`. It explains how limits like those in your screenshot (for P100, V100, A100) constrain and shape parallelization.

How GPU System Limits Affect Parallelization

This document explains how hardware limits such as **max warps per SM**, **max threads per SM**, **registers**, and **shared memory** constrain the parallelism you can actually use on GPUs like Tesla P100 (GP100), V100 (GV100), and A100 (GA100). The values below are taken from the table in your screenshot: 32 threads/warp, 64 warps/SM, 2048 threads/SM, 32 blocks/SM, 65536 registers/SM, 1024 threads/block, and 64 KB–164 KB shared memory per SM depending on architecture. ^[1]

1. Key limits per SM: what they mean

For GP100, GV100, GA100, the table lists (per SM): ^[1]

- Threads per warp: 32
- Max warps per SM: 64
- Max threads per SM: 2048
- Max blocks per SM: 32
- Max registers per SM (32-bit): 65536
- Max registers per thread: 255
- Max registers per block: 65536
- Max threads per block: 1024
- Shared memory per SM:
 - P100: 64 KB
 - V100: configurable up to 96 KB
 - A100: configurable up to 164 KB

These numbers are **upper bounds**. Your kernel's configuration (block size, registers/thread, shared memory/block) determines how close you get to those bounds and therefore how much parallelism each SM can host.

2. From limits to occupancy

2.1 Threads and warps per SM

- With 32 threads/warp and 64 warps/SM, the **theoretical maximum threads per SM** is $64 \times 32 = 2048$, which matches the “Max Threads / SM” entry.^[1]
- Occupancy cannot exceed:
 - 64 active warps
 - 2048 active threads
 - 32 active blocks

The real number of active warps/threads/blocks will usually be lower because of registers and shared memory, as described below.

2.2 Example: block size and occupancy

Suppose your kernel uses:

- Block size: 256 threads
- Registers per thread: 64
- Shared memory per block: 16 KB

On A100 (same basic SM limits as table):^[1]

1. Thread limit

- Max blocks by thread capacity:

$$\text{blocks}_{\text{threads}} = \left\lfloor \frac{2048}{256} \right\rfloor = 8$$

2. Warp limit

- Each block has $256/32 = 8$ warps.
- With 8 blocks, warps = $8 \times 8 = 64$, hitting the 64-warp limit exactly.

3. Block-count limit

- Max blocks/SM is 32, so 8 blocks is fine.

If only these limits apply, the SM can reach **100% theoretical occupancy**: 2048 threads and 64 warps. In practice, we still must check registers and shared memory.

3. Register limits and their effect

3.1 Registers per SM vs per thread

From the table: ^[1]

- **Registers per SM:** 65536 (32-bit)
- **Max registers per thread:** 255

Your kernel's actual **registers per thread** (decided by the compiler) and **block size** determine how many blocks/warps can fit before the register file is exhausted.

3.2 Example: register-limited occupancy

Assume:

- **Registers per thread:** 128
- **Block size:** 256 threads

Registers per block:

$$\text{regs/block} = 256 \times 128 = 32768$$

Max blocks limited by registers:

$$\text{blocks}_{\text{regs}} = \left\lfloor \frac{65536}{32768} \right\rfloor = 2$$

So even though threads and warps would allow 8 blocks/SM, registers allow only **2 blocks/SM**.

- **Threads per SM:** $2 \times 256 = 512$
- **Warps per SM:** $2 \times 8 = 16$

Occupancy drops to:

$$\frac{16 \text{ warps}}{64 \text{ warps max}} = 25\%$$

This shows how high register usage **reduces the number of blocks and warps per SM**, which directly reduces how much parallel work each SM and its partitions can keep “in flight”.

4. Shared memory limits and tiling

4.1 Shared memory per SM vs per block

From the table: ^[1]

- **P100:** 64 KB shared memory per SM
- **V100:** up to 96 KB per SM
- **A100:** up to 164 KB per SM

If one block uses a lot of shared memory, fewer blocks can be resident.

4.2 Example: shared-memory-limited occupancy

Assume on A100:

- Shared memory per SM: 164 KB
- Shared memory per block: 48 KB
- Block size: 256 threads

Max blocks limited by shared memory:

$$\text{blocks}_{\text{shmem}} = \left\lfloor \frac{164 \text{ KB}}{48 \text{ KB}} \right\rfloor = 3$$

So:

- 3 blocks/SM
- Threads per SM: $3 \times 256 = 768$
- Warps per SM: $3 \times 8 = 24$

Even though limits are 2048 threads and 64 warps, **shared memory restricts you to 24 warps** here. The SM partitions have fewer warps to choose from, which can hurt the ability to hide latency.

Design implication:

- Larger tiles (more shared memory per block) can improve reuse and arithmetic intensity but **lower occupancy**.
- You must balance tile size vs number of concurrent blocks per SM.

5. Max threads/block and block count

5.1 Max block size

All three GPUs have: ^[1]

- **Max thread block size:** 1024 threads

This constrains how big each block can be. You can't create a single block with more than 1024 threads, so if you need millions of threads you must use many blocks.

5.2 Max blocks per SM

- **Max thread blocks / SM:** 32 for all three GPUs. ^[1]

For very small block sizes, this becomes the limiting factor. Example:

- Block size: 32 threads (1 warp)
- Max blocks by threads: $2048/32 = 64$
- But max blocks/SM = 32, so you cap out at 32 blocks $\rightarrow$ 32 warps.

Even though the SM could host 64 warps, you only get 50% of max warps because the block-count limit kicks in.

Design implication:

- **Too small blocks** waste potential parallelism; you often want 128–256 threads/block so that you are limited by warps/threads, not by blocks/SM.

6. Comparing P100, V100, and A100

From your table:^[1]

- Many limits are identical (warps/SM, threads/SM, blocks/SM, registers/SM, registers/thread, max block size, FP32 cores/SM).
- The **big architectural difference** is shared memory per SM:
 - P100: 64 KB
 - V100: up to 96 KB
 - A100: up to 164 KB

Impact on parallelization:

- For kernels that lean heavily on shared memory tiling:
 - On P100 you might be forced to use smaller tiles per block to keep at least 2–4 blocks resident per SM.
 - On A100 you can choose **either**:
 - Much larger tiles per block (fewer blocks but more work per block), or
 - Same tile size as P100 but more blocks per SM, increasing occupancy.
- This extra shared memory on newer architectures gives you **more flexibility** in the trade-off between tile size and number of resident blocks.

7. How these limits shape your tuning decisions

When you choose `<<<grid, block>>>` and design your kernel, you are implicitly navigating these constraints:

1. **Pick a block size** (e.g., 128–256 threads) that:
 - Is a multiple of 32 (warp size).
 - Does not exceed max block size (1024).^[1]
2. **Estimate occupancy**:
 - Check threads/SM, warps/SM, blocks/SM against the limits 2048, 64, 32.^[1]
 - Then factor in register usage and shared memory per block to see the real number of resident blocks/warps.
3. **Adjust registers and shared memory**:

- If occupancy is low due to registers: simplify code or reduce per-thread state.
- If occupancy is low due to shared memory: shrink tiles or re-organize data.

4. Compare architectures (P100 vs V100 vs A100):

- Same kernel may hit different limits on each GPU because shared memory per SM differs.
- A configuration that is register-limited on P100 might become shared-memory-limited or thread-limited on A100, or vice versa.

In short, the numbers in your screenshot are the **ceiling** for how much parallelism each SM can host; your kernel's resource usage decides how close you get to that ceiling and therefore how much parallel execution the GPU can actually realize.^[1]

You can drop this text into a `gpu_parallelization_limits.md` file directly. If you'd like, I can add a small section that computes occupancy for a specific kernel you're working on (if you share registers per thread and shared memory per block).

**

1. image.jpg